让 AI agent 接入微信

通用版：任何能读文件、写代码、跑命令的 AI 编程 agent（Claude Code、Codex、Gemini CLI、Cursor 等）都可以照着做。第一部分是任务书，整段复制给你的 agent 即可开工；后面几部分是它需要的协议参考、接口约定和验收标准。

怎么用

- 1. 把第一部分「任务书」整段复制，粘贴给 agent（要在你希望它干活的那台 Windows 电脑上运行）。
- 2. agent 会先列出它要用的命令行调用方式，等你确认或它自己查证后再写代码。
- 3. 扫码绑定那一步需要你亲自在手机上操作，其他步骤它自己完成。
- 4. 验收按第七部分的清单逐条测，没测过的项目，都要求 agent 明说。

一、任务书（复制下面整段给 agent）

【目标】

帮我把「微信 ClawBot」接入这台电脑上的 AI agent。我在微信里给 ClawBot 发消息，agent 在本机执行，结果发回微信。要支持：多轮对话（会话可恢复）、进度提示、文字和文件/图片回传、/stop /new /status 等指令、定时主动推送、开机后台自启。

【硬性要求】

1. 只绑定我自己的微信账号。扫码后记录绑定用户的 ID，之后只处理这个用户的消息，其他人的消息直接忽略、不回复。

2. 不要在输出、日志或聊天里打印 bot_token、账号文件内容或任何 API key。凭据只保存在本地 state/ 目录，并加入 .gitignore。

3. 不要猜测接口字段。以本文第三部分的协议参考为准；参考里没有的、或你不确定的字段，先查证，查不到就告诉我，不要编。

4. 调用 agent 时，要把 API key 相关的环境变量从子进程里删掉，让它使用 agent 自己的登录方式。

5. 无界面运行时没法弹窗确认。危险操作要被拦下，拦下的操作列给我，等我在微信回复 /approve 后才执行。

6. 凡是对外发消息、删除或覆盖文件、付款、改账号设置，必须先在微信里问我，等我同意再做。

【交付物】

src/ilink.js：微信接口封装（请求头、收消息、发文字、上传并发文件、正在输入）。

src/agent.js：本机 agent 调用层，实现第五部分的接口约定，只写这一个文件就能换 agent。

bridge.js：常驻主程序（长轮询、白名单、消息队列、指令、本机控制口）。

login.js：生成二维码，扫码后保存绑定信息。wake.js：本机命令行工具（发文字、发文件、查状态、停止）。

config.json、README.md（写清楚怎么启动、怎么停止、怎么看日志）、test/（模拟微信服务器的端到端测试）。

【执行顺序】

第 1 步：列出你要用的 agent 的非交互调用方式：执行命令、输出格式、会话恢复、权限参数。标明哪些是你查到的，哪些是猜的。查不到就停下来问我。

第 2 步：实现 ilink.js，用模拟服务器测通收发、上传。

第 3 步：实现 agent.js，单独跑一条「只回复收到」，确认能拿到文本和会话 ID。

第 4 步：实现 bridge.js 和 wake.js，跑第七部分的验收清单，每项给出实测结果。

第 5 步：做开机自启：放进「启动」文件夹的 vbs，后台无窗口运行；程序崩溃 30 秒后自动重启。

每完成一步就汇报。扫码绑定需要我手动完成，你给出二维码后暂停等我。

【完成标准】

第七部分每一项都有实测结果。没测过的项目要明说，不能写成「应该可以」。

二、整体架构（与具体 agent 无关）

```
微信（ClawBot）
  □ 腾讯 iLink 接口：长轮询收消息 / 发消息 / 正在输入 / 上传文件
bridge.js（常驻后台）
  ├─ 白名单：只接受绑定用户
  ├─ 队列：同一批到达的消息合并成一次处理，串行执行
  ├─ 指令：/stop /new /status /model /approve ...
  ├─ 本机控制口（仅 127.0.0.1，带随机口令）：wake.js 用它发消息、发文件、停止
  └─ agent.js ──调用──> 任意 AI agent（非交互模式，支持会话恢复）
                          └─ 结果文本 + 需要发回的文件
```

- 三层分开：微信接口层（ilink.js）、桥接逻辑层（bridge.js）、agent 调用层（agent.js）。换 agent 只动第三层。
- 回复文件的约定：agent 需要把文件发回微信时，在回复里单独写一行 [[发送: 绝对路径]]。bridge 识别后上传发送，这一行不显示给用户。这个约定写在给 agent 的系统说明里。
- 主动推送：每条收到的消息都带一个 context_token（回复许可），bridge 保存到硬盘。定时任务和 wake.js 靠它主动发消息。

三、协议参考（iLink）

以下来自腾讯官方开源插件 @tencent-weixin/openclaw-weixin（MIT 协议，v2.4.10）以及已经实测通过的实现。腾讯可能随时修改接口，遇到异常先对照官方插件的源码。

接口（相对于 https://ilinkai.weixin.qq.com/）	用途	要点
POST ilink/bot/get_bot_qrcode?bot_type=3	生成登录二维码	返回 qrcode 和 qrcode_img_content（二维码内容，需渲染成图片）
GET ilink/bot/get_qrcode_status?qrcode=...	轮询扫码状态	状态：wait / scaned / confirmed / expired / need_verifycode。确认后返回 bot_token、ilink_bot_id、ilink_user_id、baseurl
POST ilink/bot/getupdates	收消息（长轮询，约 35 秒）	请求体带 get_updates_buf（上次返回的游标）；返回 msgs[] 和新的游标
POST ilink/bot/sendmessage	发消息（文字、文件、进度）	msg.to_user_id、item_list、context_token；client_id 每条都要唯一
POST ilink/bot/sendtyping	显示「正在输入」	需要 typing_ticket，先调 getconfig 获取
POST ilink/bot/getconfig	获取 typing_ticket 等配置	传 ilink_user_id 和 context_token
POST ilink/bot/getuploadurl	申请文件上传地址	传 rawsize、rawfilemd5、filesize、aeskey（hex）、media_type
POST（CDN，见下文）	上传加密后的文件	返回响应头 x-encrypted-param，作为消息里的下载参数

每个 POST 请求的公共部分

- 请求头：Content-Type: application/json；AuthorizationType: ilink_bot_token；Authorization: Bearer <bot_token>；X-WECHAT-UIN: 随机 uint32 转十进制字符串后再 base64；iLink-App-Id: bot；iLink-App-ClientVersion: 把 2.4.10 编码为 (2<<16)|(4<<8)|10，即 132106。
- 请求体都带 base_info: { channel_version: "2.4.10", bot_agent: "你的程序名/0.1" }。
- 消息 ID 是 64 位整数：JSON.parse 会丢精度。解析前先把 message_id、msg_id、svr_id 这几个字段的数字加上引号。这是最容易踩的坑。
- 返回里 ret 或 errcode 不为 0 就是出错。ret = -14 表示登录态失效，需要重新扫码。ret = -2 表示发送太频繁。

消息结构

- 入站消息关键字段：message_id、message_type（1 = 用户，2 = 机器人自己）、from_user_id、create_time_ms、item_list、context_token。
- item 类型：1 文字（text_item.text）、2 图片、3 语音（微信已转好文字）、4 文件、5 视频、11 工具调用开始、12 工具调用结束。
- 出站消息：message_type = 2，message_state = 2（完成）。同一条回复的多段，可以用同一个 run_id 归组。
- 长文本要切分，每段约 1800 字符，分多条发。

上传文件的流程（发文件和图片时用）

- 1. 读文件，计算 md5 和 rawsize。filesize = 向上取整到 16 的倍数（rawsize + 1 之后）。随机生成 16 字节 aeskey 和 16 字节 filekey。
- 2. 调 getuploadurl，拿到上传地址：upload_full_url，或者用 upload_param 拼出 CDN 地址。
- 3. 用 AES-128-ECB 加密文件内容（PKCS7 填充，Node 的 createCipheriv 默认就是），POST 到 CDN，Content-Type: application/octet-stream。最多重试 3 次；4xx 错误不重试。
- 4. 从响应头 x-encrypted-param 取下载参数。aes_key 字段填「aeskey 的 hex 字符串再做一次 base64」。
- 5. 按类型发消息：图片用 image_item，视频用 video_item，其他用 file_item（带 file_name 和 len）。单个文件上限 100 MB。

收到的文件和图片：用消息里的 encrypt_query_param 下载，再用 AES-128-ECB 解密。图片可能直接给 16 字节的 hex 密钥，优先用它。

四、参考代码（请照这个结构写，细节以你的实际测试为准）

```
// src/ilink.js 片段
import crypto from "node:crypto";
const BASE = "https://ilinkai.weixin.qq.com";
const base_info = { channel_version: "2.4.10", bot_agent: "my-agent-bridge/0.1" };

async function api(endpoint, body, token, timeoutMs = 15000) {
  const headers = {
    "Content-Type": "application/json",
    AuthorizationType: "ilink_bot_token",
    Authorization: `Bearer ${token}`,
    "X-WECHAT-UIN": Buffer.from(String(crypto.randomBytes(4).readUInt32BE(0))).toString("base64"),
    "iLink-App-Id": "bot",
    "iLink-App-ClientVersion": "132106",
  };
  const res = await fetch(new URL(endpoint, BASE + "/"), {
    method: "POST", headers, body: JSON.stringify({ ...body, base_info }),
    signal: AbortSignal.timeout(timeoutMs),
  });
  const raw = await res.text();
  if (!res.ok) throw new Error(`${endpoint} HTTP ${res.status}: ${raw.slice(0, 200)}`);
  // 消息 ID 是 64 位整数，先转成字符串，避免精度丢失
  const fixed = raw.replace(/"(message_id|msg_id|svr_id)"(\s*:\s*)(-?\d+)/g, '"$1"$2"$3"');
  return fixed ? JSON.parse(fixed) : {};
}

// 长轮询：超时属于正常情况，返回空消息
export async function getUpdates(token, buf) {
  try {
    return await api("ilink/bot/getupdates", { get_updates_buf: buf ?? "" }, token, 35000);
  } catch (e) {
    if (e.name === "TimeoutError") return { ret: 0, msgs: [], get_updates_buf: buf };
    throw e;
  }
}

export function sendText(token, to, text, contextToken) {
  return api("ilink/bot/sendmessage", {
    msg: {
      from_user_id: "", to_user_id: to,
      client_id: `bridge-${Date.now()}-${crypto.randomBytes(4).toString("hex")}`,
      message_type: 2, message_state: 2,
      item_list: [{ type: 1, text_item: { text } }],
      context_token: contextToken,
    },
  }, token);
}
```

```
// bridge.js 主循环骨架
let buf = state.buf ?? "";
const seen = new Set(state.seenIds ?? []);
for (;;) {
  const r = await getUpdates(token, buf);
  if (r.ret === -14) { await sleep(3600e3); continue; }   // 登录失效：提醒用户重新扫码
  if (r.ret && r.ret !== 0) { await sleep(2000); continue; }
  for (const m of r.msgs ?? []) {
    if (m.message_type === 2) continue;                  // 忽略机器人自己发出的
    if (seen.has(m.message_id)) continue;                // 去重
    seen.add(m.message_id);
    if (m.from_user_id !== OWNER_ID) continue;           // 白名单：只认绑定用户
    if (m.context_token) state.contextToken = m.context_token;
    queue.push(m);                                       // 串行处理，同批消息合并
    kick();
  }
  buf = r.get_updates_buf ?? buf;                        // 游标在入队后保存
  saveState({ buf, seenIds: [...seen].slice(-200), contextToken: state.contextToken });
}
```

五、agent 调用层的接口约定（这是换 agent 时唯一要改的地方）

bridge 只依赖下面这一个函数。任何 agent 只要能满足它，桥接逻辑就不用改。

```
// src/agent.js
// 输入
//   prompt       本次的用户消息（已合并、带时间戳）
//   sessionId    上一轮的会话 ID；为空表示新会话
//   cwd          工作目录
//   permissionMode 权限模式（自动拦截危险操作，不弹窗）
//   allowedTools  一次性放行的工具规则（用于 /approve）
//   systemPrompt  附加给 agent 的系统说明（写明微信场景和 [[发送: 路径]] 约定）
//   signal       AbortSignal：收到就要结束整棵进程树（相当于按 Esc）
//   timeoutMs    超时
//   onToolStart({id, name, input})  每次调用工具时回调，用于发进度
//   onToolEnd({id, isError})        工具结束时回调
// 输出
//   { ok, text, sessionId, permissionDenials: [{tool_name, tool_input}], error?, aborted? }
export async function runAgent(opts) { /* 按你的 agent 实现 */ }
```

要求	说明
非交互执行	必须能一条命令跑完并退出，而不是进入交互界面。
会话恢复	能用 ID 接着上一轮聊，否则「记得上一句」这类功能做不出来。
结构化输出	最好能输出 JSON 事件流，这样才能拿到工具调用进度、会话 ID 和最终文本。没有的话，进度功能可以先省掉。
权限控制	无界面模式下要有「自动拒绝危险操作并报告」的方式，并能一次性放行。
环境变量	运行前要删除 API key 相关变量，避免误走按量计费。
进程控制	能被整棵进程树结束（Windows 用 taskkill /T /F）。
指令文件	能读入附加的系统说明（或等价的配置方式），让它知道微信场景和文件发送约定。

常见 agent 的对应关系（以下为思路，具体参数必须以你装的版本的 --help 为准，不要照抄）

- Claude Code：非交互用 claude -p，流式输出 stream-json，恢复会话用 --resume，权限用 --permission-mode，系统说明用 --append-system-prompt-file。
- Codex CLI：查它的 exec 子命令（非交互入口）、JSON 输出选项、恢复会话的方式，以及审批和沙箱的参数。

- 其他 agent：先确认它是否有非交互模式、结构化输出和会话恢复。都没有的话，只能做「每次新会话、无进度」的简化版。

六、踩过的坑（按重要程度排序）

• 消息 ID 精度丢失

JSON.parse 直接解析会把 64 位 ID 截断，导致去重失效或回复错对象。解析前给这几个字段加引号。

• 游标保存时机不对

游标要在消息已经安全入队之后再保存。保存过早，程序崩溃时消息会丢；保存过晚，重启时会重复回复。去重集合要和游标一起持久化。

• 白名单检查放在去重之后

别人的消息记一次、然后忽略即可。无论是否来自绑定用户，游标都要照常前进，否则会卡住。

• context_token 是回复许可

回复时要带上最近一条消息的 context_token。主动推送依赖它，所以至少要先收到过你一条消息，主动推送才能发出。长时间不聊，推送可能不弹通知，用户随便发一句即可刷新。

• 发送频率限制

官方插件的注释大约是每 5 分钟 7 条。实现里保守地限制为 6 条，超了就排队等待，不要硬发。

• 同一时间只能有一个实例

两个实例会抢同一个游标，消息会被随机分给其中一个。启动时用本机端口占用来检查，占用了就直接退出并提示。

• 不要在终端里启动后台程序

用终端标签页启动的进程，关掉标签页时会被一起结束，微信就显示「暂时无法连接」。改用启动文件夹或资源管理器启动。

• 停止要结束整棵进程树

只杀主进程，agent 启动的子命令会留下来占资源。Windows 上用 taskkill /T /F。

• 合并同时到达的消息

用户连发几条时，合并成一次处理，避免启动多次 agent。合并后的消息要保留各自的时间戳。

• 提示词前缀要稳定

如果系统说明里每次都变（比如夹带当前时间），缓存命中会失效，费用可能翻几倍。时间等动态信息放在用户消息里，不要放在系统说明里。修改系统说明后，要开新会话。

• 用量数据可能是累计值

有些 agent 返回的费用是整个会话的累计值。记账时要记每轮的增量。

• 登录失效要提醒用户

ret = -14 时不要死循环重试，应通知用户重新扫码，并隔一段时间再检查。

• 文件路径和编码

Windows 上 agent
输出的路径可能带引号或反斜杠，发送前要去掉引号并检查文件存在。中文路径要确认 UTF-8 编码正常。

七、验收清单（每项都要有实测结果）

编号	测试	通过标准
1	扫码绑定	生成 state 下的绑定文件；控制台和日志里没有 token 明文
2	基本对话	微信发「你好」，出现「正在输入」，随后收到回复
3	陌生人消息	用另一个微信号发消息：没有回复，日志记录「已忽略」
4	连发三条	合并成一次回复，没有重复，没有漏掉
5	进度提示	让它执行一个命令，微信里出现进度，最终结果正确
6	/stop	10 秒内停止；它启动的子进程也被结束（用任务管理器确认）
7	/new	新会话里不记得上一轮的内容
8	会话恢复	重启程序后，继续聊，能接上上一轮
9	发文件和图片	收到对应类型；文件大小与原文件一致；图片能正常显示
10	收文件	发一个文件给它，它能读取并说出内容
11	主动推送	wake.js 发一条文字：微信收到，且不经过 agent
12	定时任务	设一个 1 分钟后触发的任务：收到消息，或判断不必打扰时不发
13	重复启动	启动第二个实例：拒绝启动并提示
14	断线恢复	断网一段时间再恢复：断线期间的消息补回复，不重复
15	崩溃恢复	强制结束主进程：30 秒内自动拉起（若做了守护）
16	开机自启	重启电脑后，无需操作即可在微信里正常聊天
17	危险操作拦截	让它删除一个测试文件：被拦下并列给你；/approve 后才执行

八、安全须知（必读）

- 这相当于把你电脑上 agent 的能力，开放给微信。只绑定你自己的账号，并在代码里强制白名单。
- 账号绑定文件等于凭据。不要分享、不要提交到代码仓库、不要贴到任何聊天或网页里。
- 权限不要设成「全部放行」。删除、覆盖、发送、付款这类操作，必须让你在微信里确认。
- 如果要给别人用，让他用自己的微信和自己的 agent 账号，各自运行，不要共用一个实例。
- 使用公司或学校提供的 agent 账号时，先确认政策允许这种自动化用途。
- 腾讯 iLink 接口没有正式的开发文档，协议来自开源插件的实现，可能随时变化。

九、已知局限

- 电脑关机时它是离线的。要全天候在线，需要一台常开的机器（比如云服务器）。
- 语音只能拿到微信的转文字结果，听不到原始语音。
- 消息长度、文件大小、发送频率都有限制，超出时需要切分或排队。
- agent 的命令行参数会随版本变化。升级 agent 后，先跑一遍验收清单。

本文档基于一次实际搭建的经验整理，不代表官方文档。接口、价格和命令参数都可能随时变化，使用前请以官方说明为准。